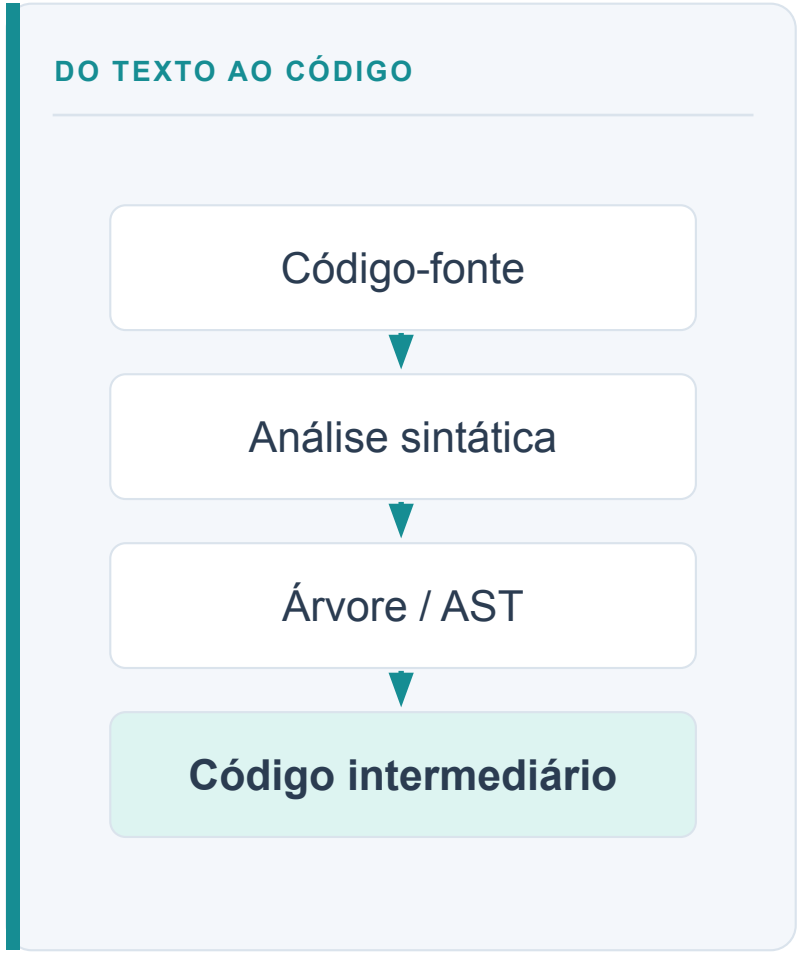
Análise sintática e formas internas

Linguagens Formais e Compiladores · Conteúdo da aula

Disciplina: Linguagens Formais e Compiladores. **Conteúdo**: análise sintática descendente e ascendente, construção de árvores sintáticas e recuperação de erros. Também estudaremos a árvore sintática abstrata e o código intermediário, conectando a gramática de uma linguagem à representação que o compilador utiliza para produz*ir* programas executáveis.



Objetivos de aprendizagem

O que o estudante deverá compreender e aplicar

Distingu*ir* análise descendente de ascendente, acompanhar derivações, interpretar árvores sintáticas e reconhecer estratégias de recuperação de erros. Também deverá explicar a diferença entre árvore concreta e árvore abstrata, além de transformar uma expressão simples em instruções de código intermediário.

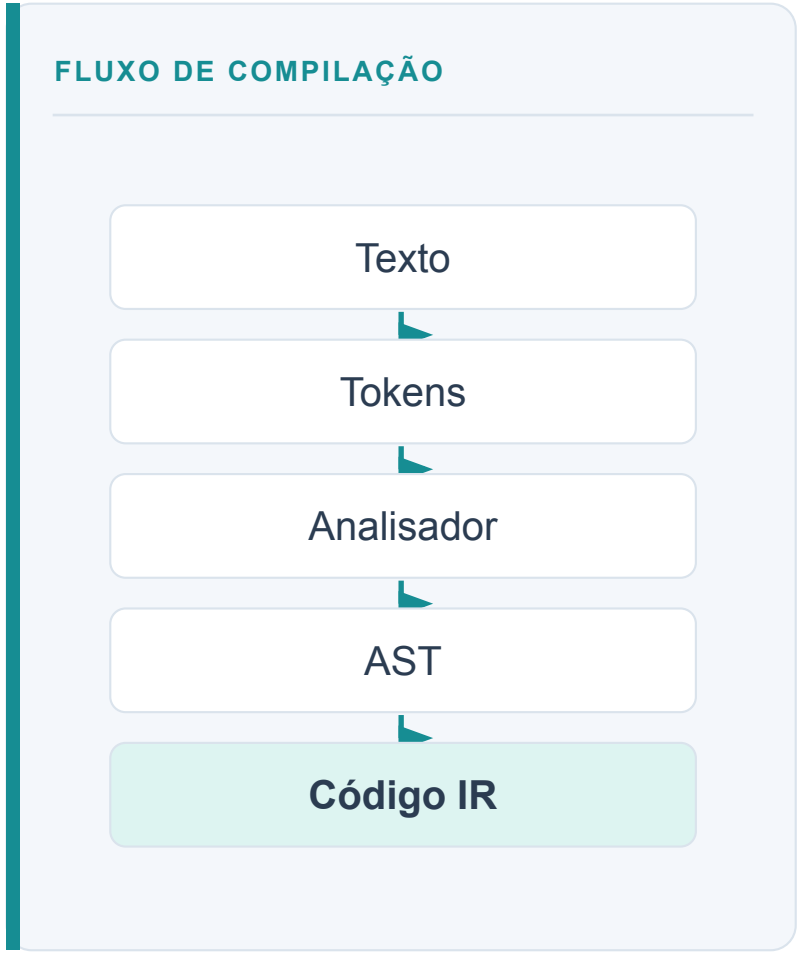
QUATRO COMPETÊNCIAS

- **Comparar analisadores**
- Ler árvores sintáticas
- Diagnosticar erros
- Construir AST e código

Onde entra o analisador sintático?

Uma etapa da arquitetura de compiladores

Um compilador costuma divid*ir* o processamento em etapas. Prime*iro*, a análise léxica converte caracteres em unidades chamadas *tokens*. Depois, a análise sintática verifica como essas unidades podem ser combinadas. Uma árvore resultante auxilia a análise semântica e, posteriormente, a geração de uma representação intermediária.



Relembrando a gramática livre de contexto

A gramática define estruturas permitidas

Uma gramática livre de contexto descreve como expressões e comandos podem ser formados. Seus símbolos terminais aparecem na entrada, enquanto os não terminais representam estruturas que ainda podem ser expandidas. As produções indicam substituições válidas. O analisador sintático utiliza essas regras para reconhecer a organização do programa.

GRAMÁTICA DE EXEMPLO

```
E → E + T | T

T → T * F | F

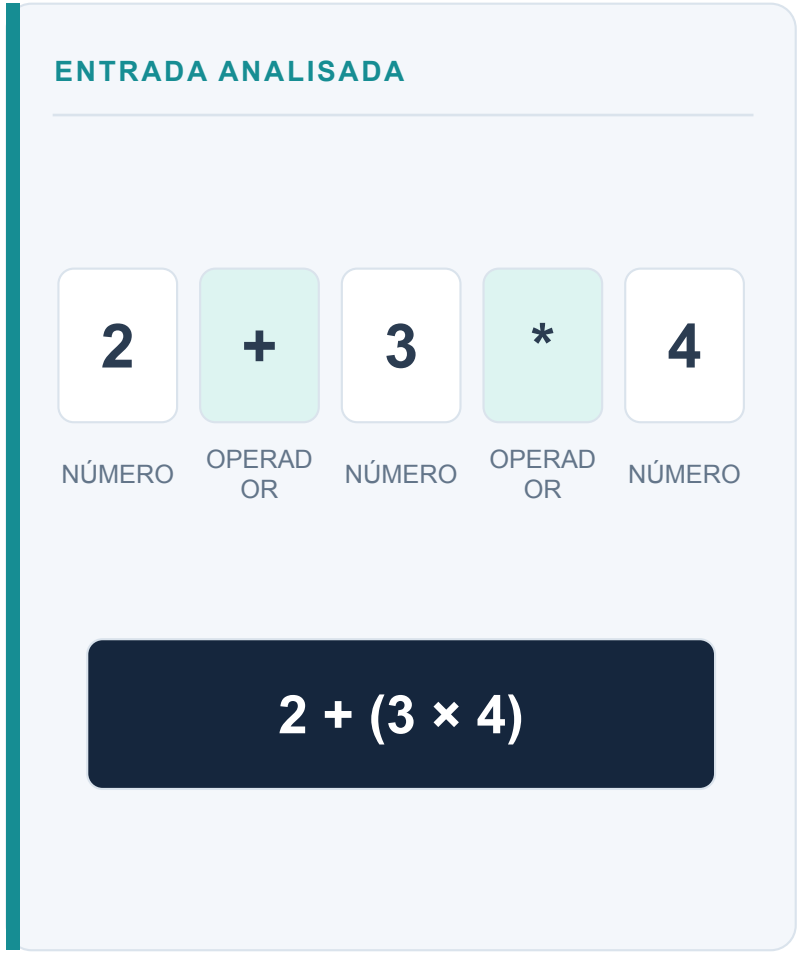
F → ( E ) | n
```

n representa um número

Da sequência de *tokens* à estrutura

O papel dos símbolos reconhecidos

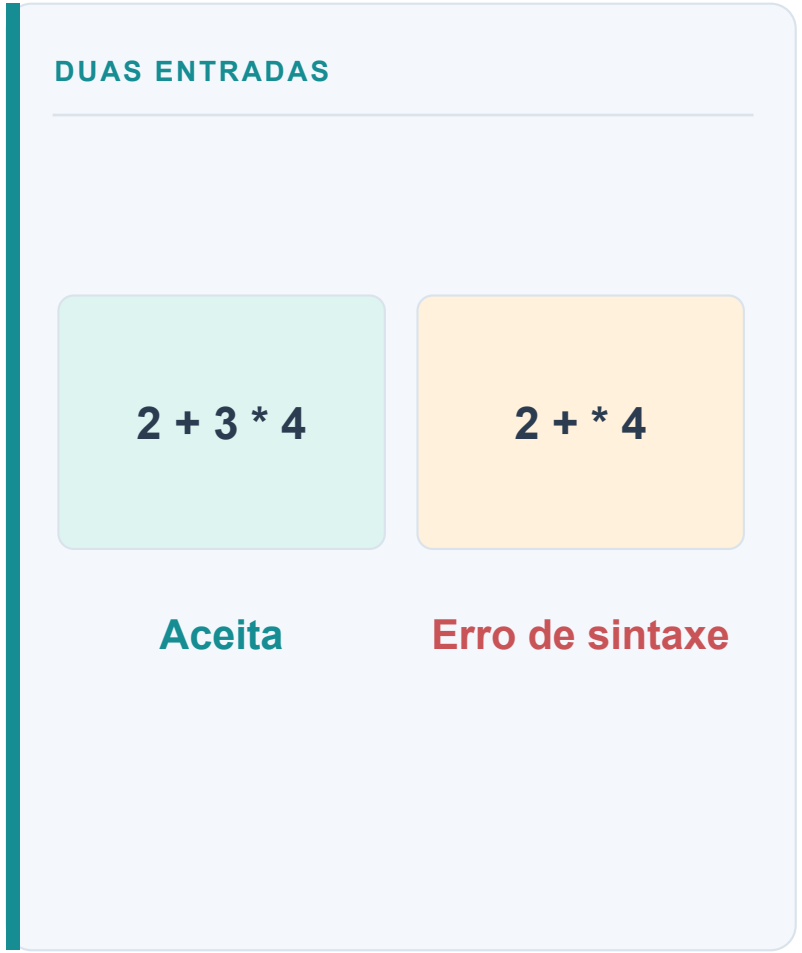
Considere a expressão 2 + 3 * 4. O analisador léxico pode produz*ir* a sequência NÚMERO, MAIS, NÚMERO, VEZES, NÚMERO. O analisador sintático verifica se essa ordem é aceita pela gramática e determina quais operadores se associam aos operandos, respeitando a precedência definida.



O que significa analisar sintaticamente?

Reconhecimento e construção de estrutura

Analisar sintaticamente é decid*ir* se uma sequência de *tokens* segue as regras de uma gramática e, quando necessário, constru*ir* sua estrutura. Uma expressão pode ter símbolos válidos isoladamente e ainda estar incorreta. Por exemplo, 2 + * 4 contém operadores em uma ordem incompatível com a gramática apresentada.



Duas estratégias de análise

Descendente e ascendente

Existem duas grandes famílias de analisadores. Na abordagem descendente, parte-se do símbolo inicial e tenta-se explicar a entrada por expansões. Na abordagem ascendente, parte-se dos *tokens* e procura-se agrupá-los até chegar ao símbolo inicial. Ambas reconhecem estruturas, mas organizam a busca de mane*ir*as diferentes.



Análise descendente

Começar pela regra inicial

Um analisador descendente inicia no símbolo inicial da gramática e escolhe produções para reproduz*ir* a sequência de entrada. Cada escolha antecipa qual estrutura se espera encontrar. Essa estratégia é intuitiva e pode ser implementada por funções recursivas, mas certas gramáticas exigem adaptações antes de serem utilizadas.

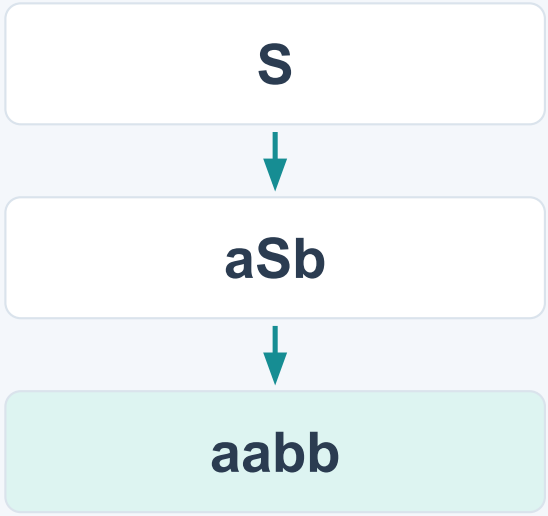


Derivação mais à esquerda

Exemplo com uma gramática simples

Considere a gramática S → aSb | ab. Para reconhecer aabb, podemos começar por S, aplicar a produção aSb e depois substitu*ir* o S restante por ab. A sequência obtida é aabb. A derivação mais à esquerda sempre expande prime*ir*o o não terminal situado mais à esquerda.

PASSOS DA DERIVAÇÃO



S ⇒ aSb ⇒ aabb

Descida recursiva na prática

Uma função para cada não terminal

Na descida recursiva, o programa organiza a gramática em funções que chamam umas às outras. Para analisar um fator, por exemplo, podemos aceitar um número ou abr*ir* parênteses, analisar outra expressão e exig*ir* o fechamento. A técnica é didática porque aproxima regras gramaticais de estruturas conhecidas de programação.

PSEUDOCÓDIGO DE F

```
fator():
  se próximo = número: consumir
  senão se próximo = "(":
    consumir; expressão()
    exigir ")"
  senão: informar erro
```

Analisadores *LL(1)*

Escolher uma produção com antecipação

A sigla *LL(1)* descreve um analisador que lê a entrada da esquerda para a d*ir*eita, constrói uma derivação mais à esquerda e consulta um *token* de antecipação para decid*ir* a regra. Esse *token* futuro recebe o nome de *lookahead*. Para funcionar sem conflitos, a gramática precisa ser adequada a essa escolha.

INTERPRETANDO LL(1)

L: leitura da esquerda

L: derivação à esquerda

1: um token adiante

FIRST e *FOLLOW*: ideia intuitiva

Conjuntos que orientam a previsão

O conjunto *FIRST* informa quais terminais podem aparecer no início de uma sequência derivada de um símbolo. Já *FOLLOW* informa quais terminais podem surg*ir* imediatamente depois de um não terminal em uma derivação. Esses conjuntos ajudam a montar tabelas de análise preditiva e a identificar decisões ambíguas.

EXEMPLO DIDÁTICO

Conjunto	Interpretação
FIRST(F)	n ou (
FOLLOW(F)	+, *,) ou fim

Exemplo ilustrativo para F → n | (E)

Recursão à esquerda

Por que certas regras dificultam a descida?

Uma regra como E → E + T | T permite que a própria expressão reapareça imediatamente à esquerda. Um analisador descendente recursivo ingênuo pode chamar a função E sem consum*ir* nenhum *token*, repetindo indefinidamente. Para evitar isso, reescrevemos a gramática preservando a linguagem e reorganizando suas produções.

TRANSFORMAÇÃO

ANTES

$$E \rightarrow E + T \mid T$$

DEPOIS

$$E \rightarrow T\ R$$
$$R \rightarrow +\ T\ R \mid \varepsilon$$

ε representa a cadeia vazia

Análise ascendente

Reduzir tokens até o símbolo inicial

Na abordagem ascendente, o analisador começa com os *tokens* e tenta reconhecer trechos que correspondem ao lado d*ir*eito de produções. Cada trecho reconhecido pode ser reduzido ao não terminal correspondente. O processo continua até formar o símbolo inicial, reproduzindo uma derivação mais à d*ir*eita em ordem inversa.



Empilhar e reduz*ir*

Ações de um analisador ascendente

Um analisador do tipo deslocamento e redução mantém uma pilha com símbolos já processados. A ação deslocar transfere o próximo *token* da entrada para a pilha. A ação reduz*ir* substitui um conjunto de símbolos por um não terminal quando alguma produção é reconhecida. O objetivo é aceitar a entrada completa.

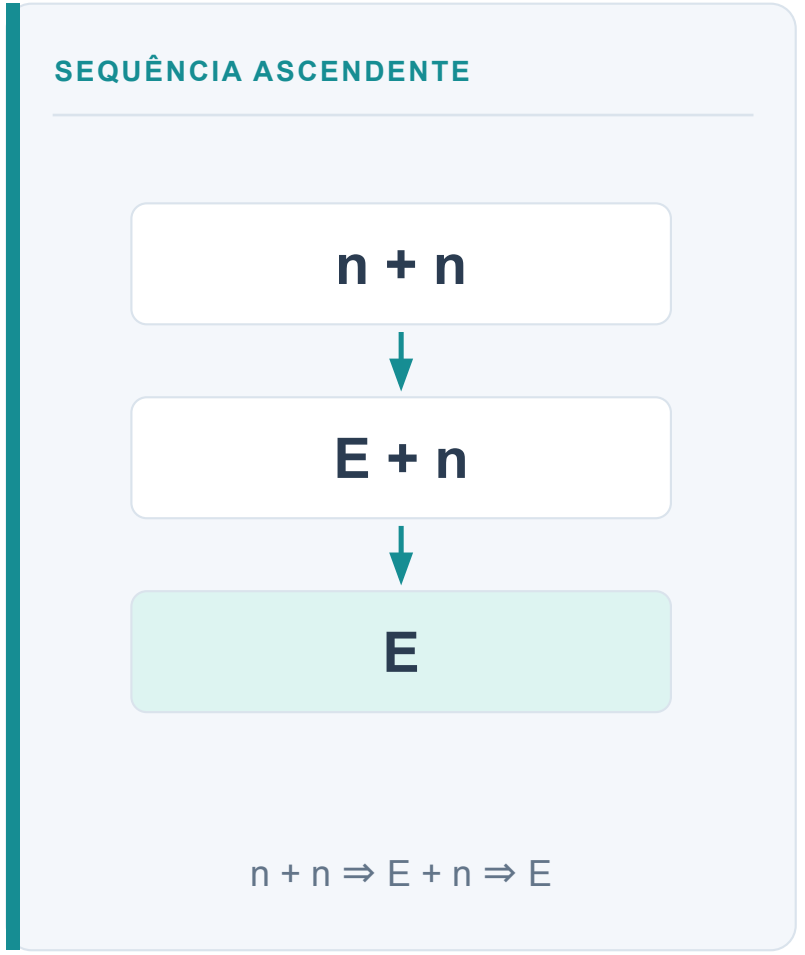
EXEMPLO SIMPLIFICADO

Pilha	Entrada	Ação
∅	n + n	deslocar
n	+ n	reduzir F → n
F	+ n	continuar

Uma redução passo a passo

Reconhecendo uma expressão curta

Considere a gramática E → E + n | n. Para analisar n + n, primeiro reconhecemos o número inicial como uma expressão E. Depois, combinamos E, o operador + e o número seguinte, aplicando E → E + n. Ao final, toda a entrada foi reduzida ao símbolo E.



LR e conflitos de decisão

Quando a escolha não é imediata

Analisadores *LR* combinam leitura da esquerda para a d*ir*eita com construção ascendente. Em algumas gramáticas, pode surg*ir* dúvida entre deslocar o próximo *token* ou reduz*ir* um trecho já reconhecido. Também pode haver mais de uma redução possível. Esses conflitos indicam necessidade de ajustar regras ou prioridades.

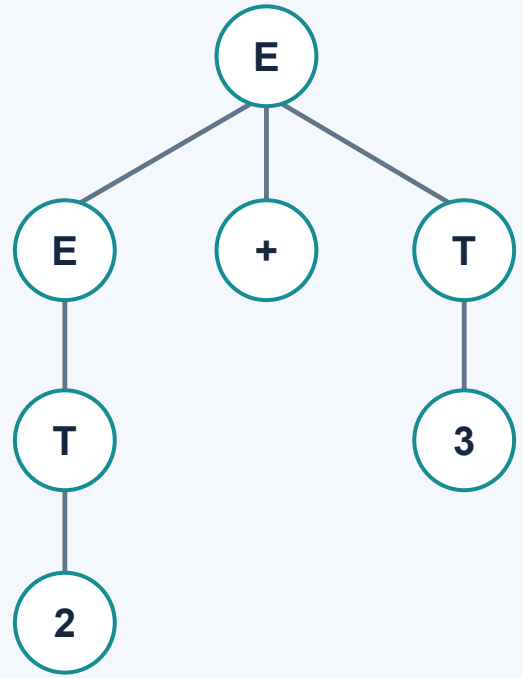


Árvore sintática concreta

Representar uma derivação completa

A árvore sintática concreta registra a aplicação das produções da gramática. Os nós internos representam não terminais e as folhas correspondem aos terminais da entrada. Essa representação ajuda a visualizar agrupamentos e derivações, embora possa conter muitos nós que servem apenas à descrição gramatical.

ÁRVORE CONCRETA SIMPLIFICADA

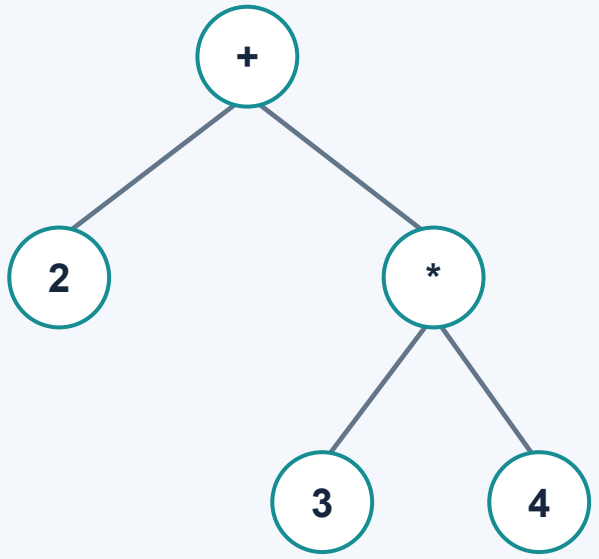


Árvore para 2 + 3 * 4

Precedência e agrupamento

Uma árvore sintática permite enxergar a hierarquia dos operadores. Na expressão 2 + 3 * 4, a multiplicação deve aparecer em uma subárvore abaixo da soma. Assim, primeiro se forma o resultado de 3 * 4 e depois se adiciona 2. A estrutura corresponde ao valor 14.

HIERARQUIA DE OPERADORES



Ambiguidade sintática

Mais de uma árvore para a mesma entrada

Uma gramática é ambígua quando alguma sequência válida admite mais de uma árvore sintática distinta. Isso pode alterar o significado de uma expressão, como ocorre quando soma e multiplicação não possuem precedência definida. O projeto da gramática deve reduz*ir* interpretações concorrentes e tornar as escolhas explícitas.

SEM REGRAS DE PRECEDÊNCIA

(2 + 3) × 4

20

2 + (3 × 4)

14

O que é um erro de sintaxe?

Quando a estrutura esperada não aparece

Um erro sintático ocorre quando a sequência de *tokens* não pode ser explicada pelas regras adotadas. Parênteses sem fechamento, operadores sem operandos e comandos incompletos são exemplos frequentes. Um bom analisador precisa localizar o problema e informar o que era esperado, evitando mensagens vagas que dificultem a correção.

DIAGNÓSTICO ILUSTRATIVO

```
resultado = (2 + 3 * ;
```

Esperado: número ou “(” após *

Recuperação de erros: modo pânico

Pular até um ponto de sincronização

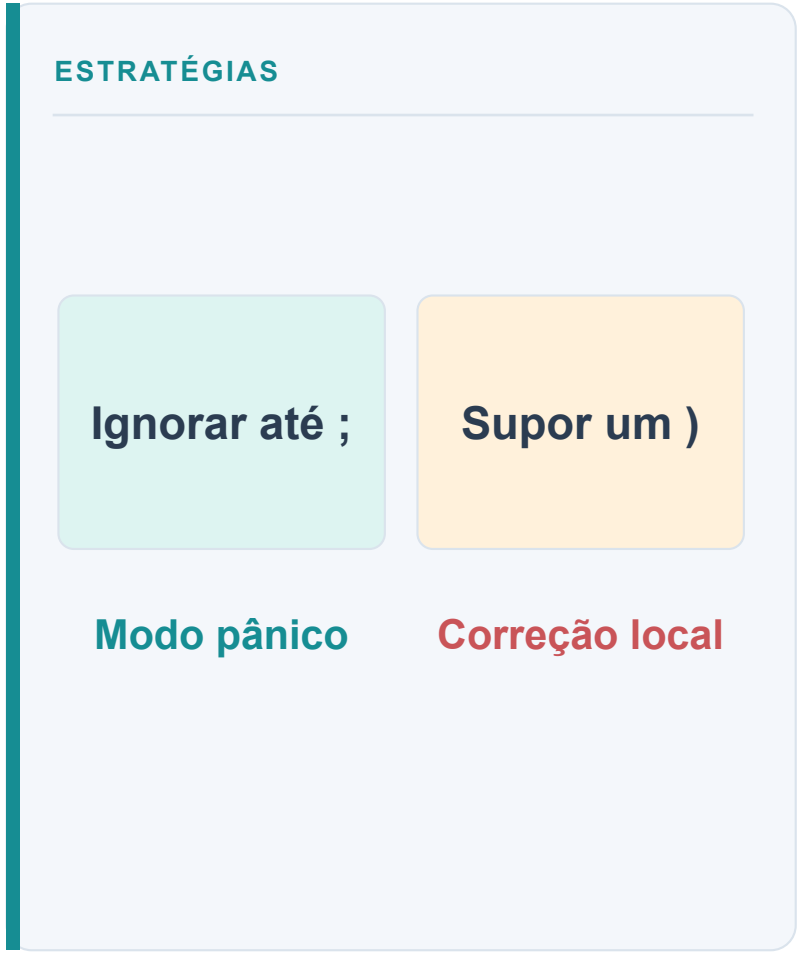
Na recuperação por modo pânico, o analisador descarta *tokens* até encontrar um símbolo considerado seguro para retomar a análise. Em linguagens de programação, ponto e vírgula ou fechamento de bloco podem funcionar como marcadores de sincronização. A vantagem é evitar que um erro impeça o exame do restante do programa.



Recuperação local e mensagens úteis

Continuar com cautela

Outra estratégia consiste em realizar uma correção local, como supor a inserção de um *token* ausente ou ignorar um símbolo inesperado. Essa tentativa pode permit*ir* a identificação de novos erros, mas precisa evitar diagnósticos em cascata. O objetivo principal é ajudar o programador, sem mascarar problemas reais.

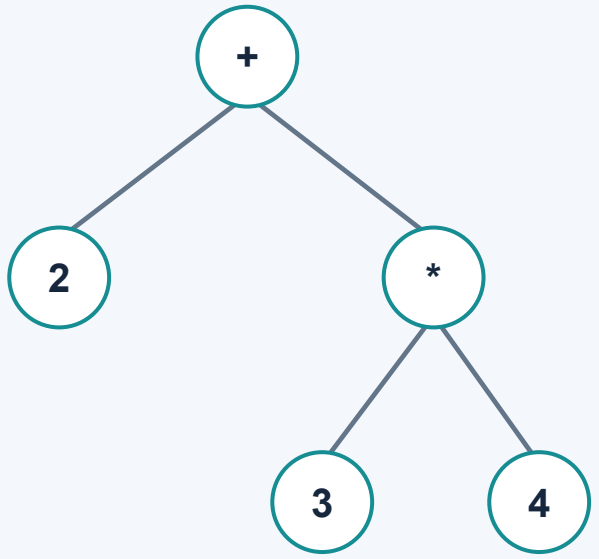


O que é uma *AST*?

Árvore sintática abstrata

A árvore sintática abstrata, conhecida pela sigla *AST*, representa a estrutura essencial de um programa. Ela conserva operadores, operandos e construções relevantes, mas costuma eliminar detalhes como parênteses de agrupamento e nós auxiliares da gramática. Por isso, é uma forma interna mais conveniente para análise semântica e geração de código.

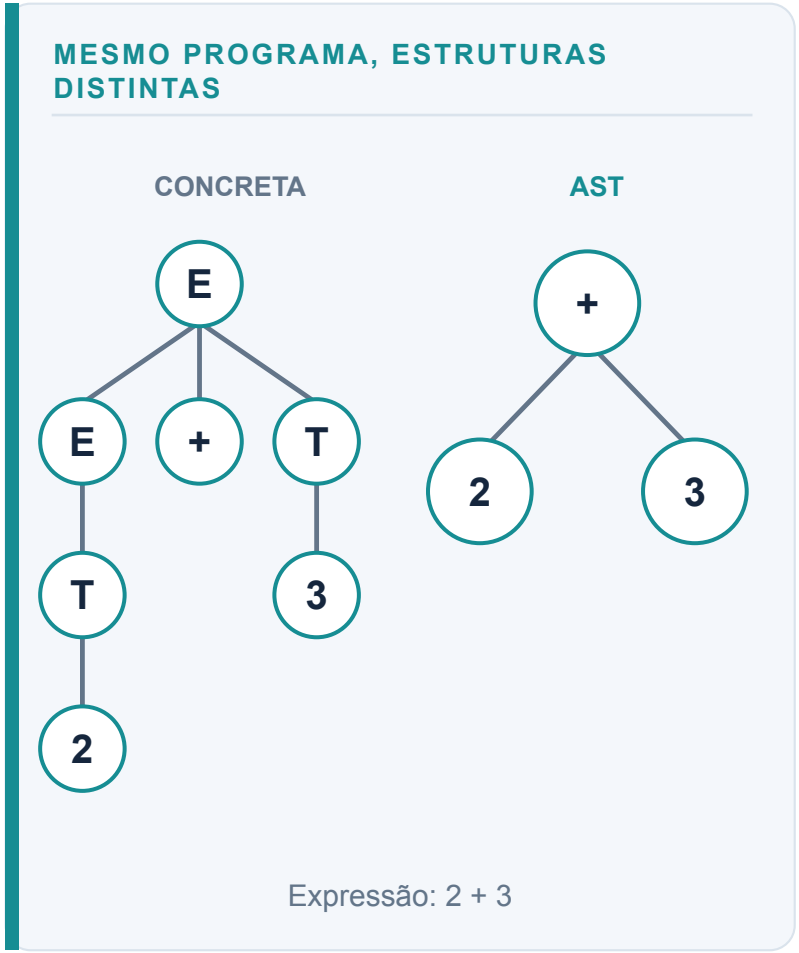
AST DE UMA EXPRESSÃO



Árvore concreta versus *AST*

Duas representações com objetivos diferentes

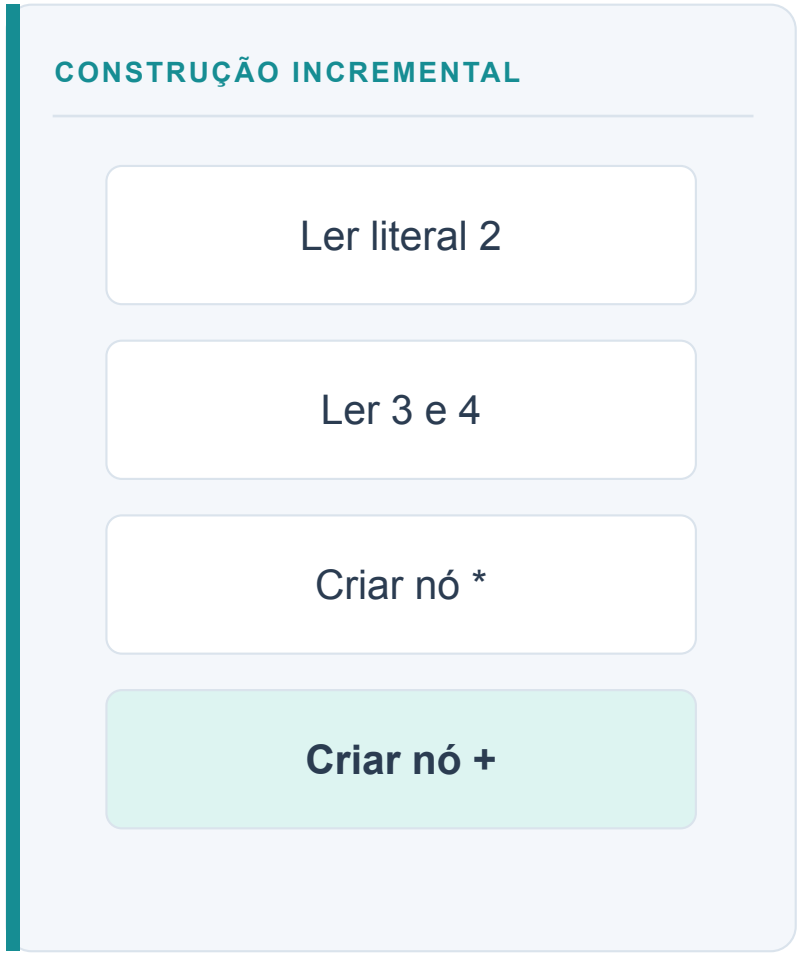
A árvore concreta acompanha de perto as regras da gramática, inclusive nós intermediários e símbolos de pontuação. A *AST* concentra a estrutura necessária às etapas seguintes. Ambas podem representar a mesma expressão corretamente, mas a *AST* costuma ser menor e mais prática para verificar tipos, transformar operações e gerar código.



Como construir uma *AST*?

Criar nós durante a análise

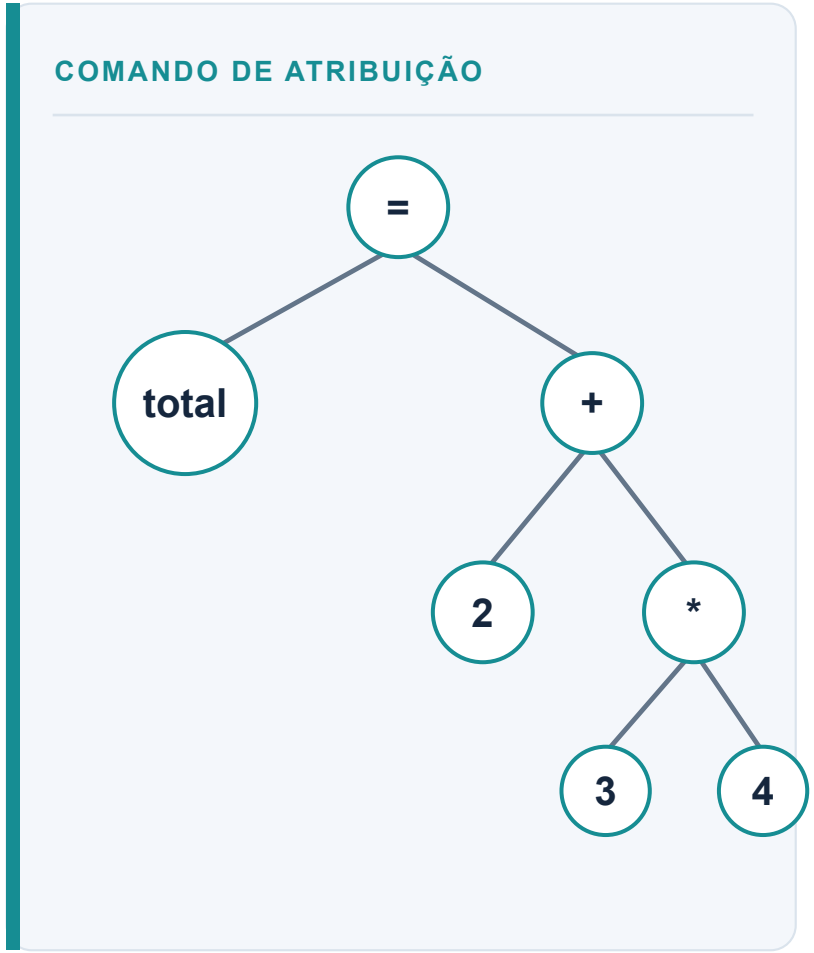
Durante a análise sintática, cada produção reconhecida pode produzir um nó apropriado. Um número origina um nó literal; uma soma cria um nó operador com filhos esquerdo e direito. Ao combinar esses nós, formamos a *AST*. O processo associa regras gramaticais a estruturas de dados reutilizáveis.



AST de um comando de atribuição

Representar estruturas além de expressões

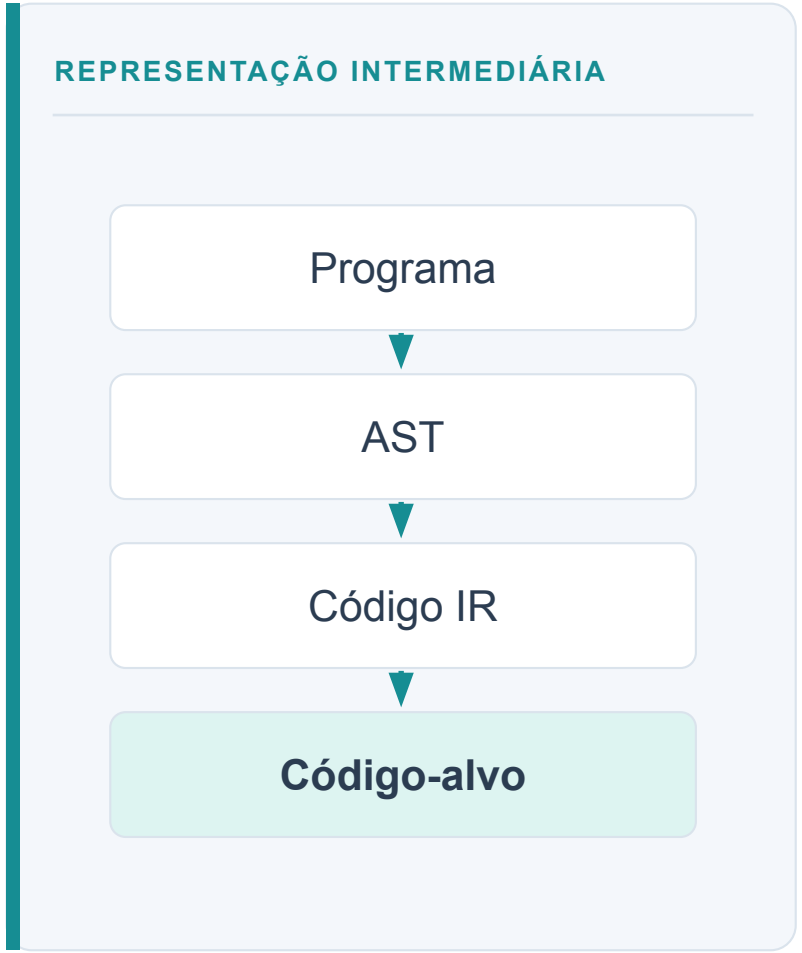
Uma *AST* também pode representar comandos completos. Na atribuição total = 2 + 3 * 4, a raiz pode ser um nó de atribuição, com o identificador total de um lado e a expressão aritmética do outro. Isso permite ao compilador distingu*ir* a operação de armazenar um valor do cálculo que a precede.



Por que gerar código intermediário?

Separar análise de tradução final

O código intermediário é uma representação que fica entre o programa-fonte e o código de máquina ou outra saída. Ele facilita transformações, análises e otimizações sem depender d*ir*etamente de um processador específico. Uma mesma representação pode serv*ir* de base para diferentes destinos, desde que exista um tradutor apropriado.



Código de três endereços

Quebrar expressões em operações simples

Uma forma didática de código intermediário é o código de três endereços. Cada instrução geralmente combina até dois operandos e um destino. Expressões complexas são divididas em operações menores, usando variáveis temporárias. Essa representação torna explícita a ordem de cálculo e simplifica algumas otimizações locais.

PARA 2 + 3 * 4

```
t1 = 3 * 4

t2 = 2 + t1

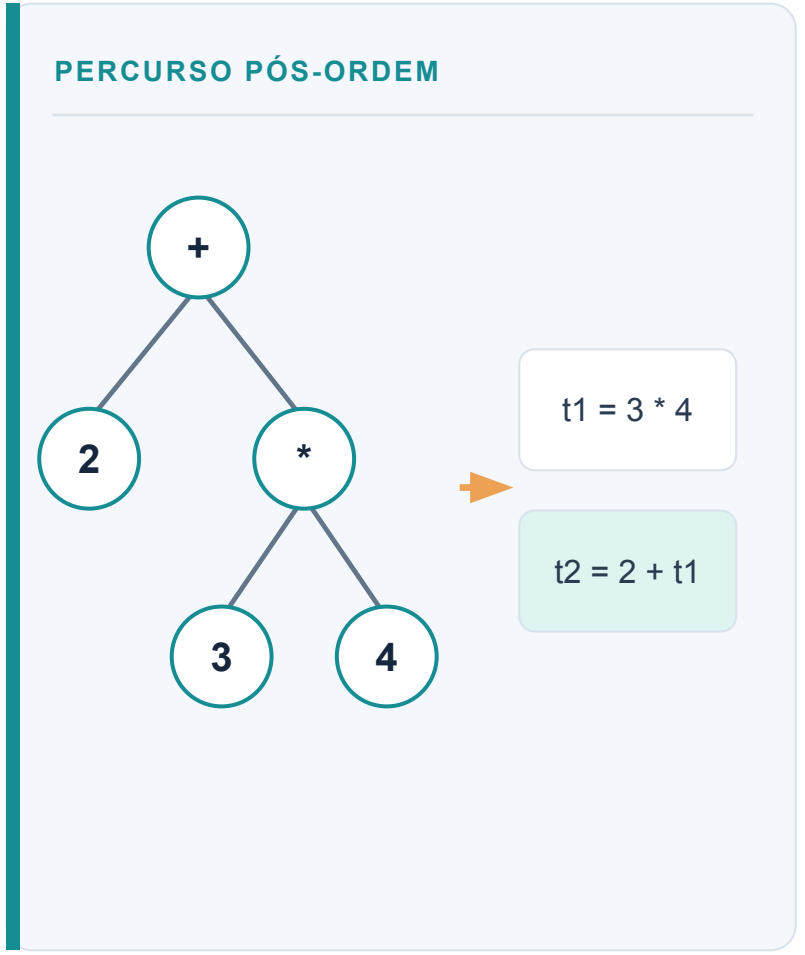
resultado = t2
```

t1 e t2 são temporários

Da *AST* ao código intermediário

Percorrer os nós na ordem correta

Para gerar instruções a part*ir* de uma *AST*, podemos visitar prime*iro* as subárvores que produzem os operandos. Em seguida, emitimos a operação do nó atual e guardamos seu resultado em uma variável temporária. Esse percurso preserva a hierarquia representada pela árvore e prepara o programa para fases posteriores.



Ferramentas para a aula prática

JFLAP 7.0 e Python

O *JFLAP* permite cad*astr*ar gramáticas livres de contexto, testar cadeias e observar derivações e árvores. Para constru*ir* uma *AST* e emit*ir* código de três endereços, utilizaremos um pequeno programa em *Python*, que funciona sem bibliotecas externas. As ferramentas são complementares: uma destaca a teoria formal e a outra a implementação.

ESCOLHA POR FINALIDADE

- **JFLAP: gramáticas**
- JFLAP: derivações
- Python: AST
- Python: código intermediário

Laboratório *JFLAP*: preparar a gramática

Primeiros passos no aplicativo

Abra o *JFLAP* 7.0 e escolha a opção *Grammar*. Insira as produções S → aSb e S → ab, uma por linha. Confira se o símbolo inicial é S e salve a gramática. Essa linguagem permite observar o aninhamento de letras a e b, uma estrutura adequada para estudar derivações.

PASSOS NO JFLAP

1. Abrir Grammar
2. Inserir S → aSb
3. Inserir S → ab
4. **Salvar arquivo .jff**

Laboratório *JFLAP*: testar e interpretar

Executar o analisador da ferramenta

Com a gramática aberta, acesse o menu *Input* e selecione *Brute Force Parse*. Digite aabb e execute a análise. Observe a derivação S ⇒ aSb ⇒ aabb e a árvore exibida. Depois teste abab e explique por que a sequência não pertence à linguagem gerada pelas regras escolhidas.

TESTE ORIENTADO

- Input → Brute Force Parse
- Testar: aabb
- Observar a derivação
- **Testar: abab (rejeitada)**

Laboratório *Python*: executar a *AST*

Da expressão ao código de três endereços

Abra o arquivo analisador_expressoes.py no *VS Code* ou em outro editor e execute *python* analisador_expressoes.py. O programa reconhecerá números, operadores e parênteses, construirá uma *AST* e imprimirá o código intermediário. Modifique a expressão de teste e compare a árvore, as instruções geradas e o resultado calculado.

EXECUÇÃO ESPERADA

```
$ python analisador_expressoes.py

Entrada: 2 + 3 * 4

AST: (+ 2 (* 3 4))

t1 = 3 * 4

t2 = 2 + t1

Resultado: 14
```

Exercício resolvido

Construir AST e código intermediário

Considere a expressão 5 + 2 * 3. Prime*iro*, reconhecemos que a multiplicação possui precedência maior e formamos a subárvore 2 * 3. Depois, criamos a raiz de soma com o número 5. O código intermediário calcula t1 = 2 * 3 e t2 = 5 + t1, obtendo 11.

SOLUÇÃO COMPLETA

5 + 2 * 3

```
t1 = 2 * 3

t2 = 5 + t1

valor final = 11
```

Exercício avaliativo

Aplicação individual ou em dupla

Analise a expressão (4 + 2) * 3 e responda: qual é a operação da raiz da *AST*? Desenhe a árvore, gere o código de três endereços e calcule o resultado. Depois, explique uma diferença entre análise descendente e ascendente e indique uma estratégia para recuperar erros sintáticos.

ENTREGÁVEIS

1. Desenho da AST
2. Instruções temporárias
3. Resultado numérico
4. Explicações conceituais

Síntese da aula

Como os conceitos se conectam

A gramática define a forma das construções válidas. O analisador descendente expande regras; o ascendente reconhece e reduz trechos. Árvores concretas documentam derivações, enquanto a *AST* conserva a estrutura essencial. A recuperação de erros melhora os diagnósticos, e o código intermediário prepara a tradução para etapas posteriores.



Referências e recursos

Bibliografia e documentação de apoio

O estudo foi organizado a part*ir* do plano de ensino de Linguagens Formais e Compiladores e de materiais didáticos sobre gramáticas livres de contexto. Para aprofundamento, consulte os livros indicados na disciplina, o tutorial oficial do *JFLAP* e a documentação do LLVM sobre construção de árvores abstratas e geração de código.

FONTES PRINCIPAIS

SANTOS; LANGLOIS. Compiladores (2018).

COOPER; TORCZON. Construindo compiladores (2014).

RODGER; FINLEY. JFLAP (2005/2006).

JFLAP: jflap.org/tutorial/frameindex.html

LLVM: llvm.org/docs/tutorial/